

A.E.T.H.E.R — RUNBOOK: a working demo on the 12th

Build days: 9, 10, 11 June. Demo: 12 June. This is the *fastest reliable path*, not the maximal one. It is built so that **by end of Day 1 you already have a live, working ROS2 demo** (the replay path), and everything after that strengthens it. Front-load the risky steps; never bet the demo on the one uncertain step.

The three tiers (build them in this order — each is a complete fallback for the next)

Tier	What runs	Depends on	Confidence
T1 — Integrity live (GUARANTEED)	<code>replay_node</code> → the REAL <code>integrity_monitor</code> + <code>degradation_manager</code> + <code>health_cockpit</code> . Kill-camera → trust RED < 0.5 s → bound blooms → recover, in RViz + HUD.	only ROS2 (no OpenVINS, no Gazebo)	Very high
T2 — Real VIO accuracy	OpenVINS on the EuRoC dataset → real drift < 1.5%, estimate-vs-ground-truth plots.	ROS2 + OpenVINS (out-of-box on EuRoC)	High
T3 — Full sim (the bonus)	Gazebo tunnel + drone flying → OpenVINS on the sim feed → integrity live on real sim VIO.	+ Gazebo + OpenVINS convergence on custom sim	Medium

Your demo = T1 (the visual wow) + T2 (the real numbers). T3 is upside. If T3 lands you show it; if not, you are still complete and honest: "integrity system live (T1) + measured VIO accuracy on real flight data (T2); full sim integration in progress."

DAY 1 (9 Jun) — environment + T1 working by tonight

Morning — Ubuntu + ROS2 (the #1 risk, do it first). 1. Boot Ubuntu 22.04 (dual-boot). If you don't have it: install it now (USB, ~1 hr). No Ubuntu box by noon → spin a **cloud GPU instance** (Lambda/Paperspace, Ubuntu 22.04 image) and work there. 2. `git clone https://github.com/TheClazer/A.E.T.H.E.R.git` `aether` && `cd aether` 3. `chmod +x scripts/*.sh && ./scripts/setup_ubuntu.sh` (ROS2 Humble + Gazebo Harmonic + OpenVINS build). Add the two `source` lines to `~/.bashrc`.

Afternoon — get T1 live (this is your safety demo, lock it today).

```
cd ~/aether
./scripts/run_replay_demo.sh      # builds + launches replay + integrity stack
# new terminal:
rviz2 -d src/aether_bringup/config/aether.rviz
ros2 topic echo /nav/state        # NOMINAL
./scripts/run_demo.sh kill        # -> /nav/state INERTIAL within ~0.3 s, ellipse blooms
./scripts/run_demo.sh restore     # -> recovers to NOMINAL
```

 **GATE 1 (end of Day 1):** the kill-camera beat plays live in RViz + the trust topic flips RED < 0.5 s. **If green, you already have a working demo for the 12th.** Record a screen-capture as the rung-3 backup now.

```
Troubleshooting: build aether_msgs first ( colcon build --packages-select aether_msgs ); if sensor_msgs_py is missing, sudo apt
install ros-humble-sensor-msgs-py .
```

DAY 2 (10 Jun) — T2: real VIO accuracy on EuRoC

1. Get a EuRoC ROS2 bag (e.g. **V1_01_easy**, **MH_03**). (ASL EuRoC → convert to ROS2 bag, or use a pre-made ROS2 bag.)
2. Run OpenVINS on it (out-of-box config):

```
source ~/ws_ov/install/setup.bash
ros2 launch ov_msckf subscribe.launch.py config:=euroc_mav &
ros2 bag play V1_01_easy      # publishes /cam0/image_raw /cam1/image_raw /imu0
ros2 bag record /ov_msckf/odomimu # record the estimate
```

3. Evaluate the real drift:

```
python eval/odom_to_tum.py <rec_bag> /ov_msckf/odomimu est.txt
# EuRoC ground truth -> gt.txt (from the dataset's state_groundtruth csv)
python eval/compute_drift.py gt.txt est.txt          # expect < 1.5% / 200 m
```

 **GATE 2:** real drift < 1.5% and an estimate-vs-GT plot exist. **Now T2 (numbers) + T1 (live integrity) = a complete DP7 submission.** Tag: `git tag floor-locked && git push origin floor-locked`.

Bonus (if ahead): run the integrity stack on the *live OpenVINS-on-EuRoC* output instead of the replay — point `integrity_monitor` at the real `/ov_msckf/odomimu`. If OpenVINS keeps propagating through a blanked-camera stretch, you get the real-data kill-camera beat; if its outage handling is finicky, keep the replay path for the live beat (that's why T1 exists).

DAY 3 (11 Jun) — T3 sim (bonus) + polish

1. Sim setup (DP7 deliverable #1):

```
gz sim sim/worlds/tunnel.sdf          # tunnel + drone load; sensors + /aether/ground_truth stream
ros2 launch aether_bringup sim_world.launch.py
```

Get the drone flying a ~200 m pass (PX4 SITL `make px4_sitl gz_x500`, or a scripted velocity/pose). Verify `/camera/*` and `/imu/data` and `/aether/ground_truth` are publishing. 2. **(Stretch) OpenVINS on the sim feed:** uncomment the `ov_msckf` node in `vio.launch.py`, set the camera/IMU calibration from the SDF, and tune until it converges. If it converges → full T3. If it fights → stop; you have T1+T2. **Do not let this eat the demo.** 3. **Polish:** rehearse the 60-second beat to 5/5 clean runs. Finalize the backup video. `git tag demo-final && git push origin demo-final`.

12 JUN — the 3-hour final

- Boot Ubuntu, `cd aether`, `colcon build --symlink-install`, `source install/setup.bash`.
- **Live:** `./scripts/run_replay_demo.sh` + RViz + the HUD → the kill-camera beat (T1).
- **Numbers:** show the EuRoC drift plot + the offline figures (T2 + `docs/figures`).
- **Sim:** show the Gazebo tunnel + drone (T3, even if VIO-on-sim is "in progress").
- If anything stutters live → drop to the recorded backup video. The beat is identical.

Say it honestly: "Integrity system running live; VIO accuracy measured on real EuRoC flight data; full Gazebo-VIO integration in progress." The spine, three times: *"When the camera dies, our drift bound still covers the true position 95%+ of the time — and we know within half a second."*

The one rule

Build **T1 → T2 → T3 in that order**. After Day 1 you have a live working demo; after Day 2 you have a complete, honest DP7 submission; Day 3 is pure upside. You are never in a position with "nothing to show."